VINUNIVERSITY

# Monitoring, Logging & Observability

*AICB-P1 · Ngày 13 · Biết agent đang hoạt động thế nào*

**Tên Giảng Viên**

VinUniversity · Phase 1 · 2026

## HÃY SUY NGHĨ...

*“3 câu chuyện có thật: CFO hỏi vì sao hóa đơn OpenAI tháng này $45,000 — không ai trả lời được. Chatbot ngân hàng viral lên Twitter vì trả lời xúc phạm — phát hiện sau 6 giờ. Agent chạy “bình thường” nhưng engagement giảm 30% suốt 2 tuần — không alert nào fire.”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

# Nội Dung Bài Học

1. Foundations: Observability & 3 pillars
2. AI metrics deep dive (TTFT, quality, cost, drift)
3. Structured logging (schema, PII, aggregation)
4. Distributed tracing (OpenTelemetry, Langfuse)
5. SLO-based alerting & dashboard design
6. Tools landscape 2026 + case studies
7. Lab 13: monitoring blueprint cho agent
8. Preview Day 14: Evaluation with RAGAS

# Mục Tiêu Ngày 13 (Bloom's Taxonomy)

- **Remember** — liệt kê được 3 pillars of observability và 6 loại AI-specific metrics
- **Understand** — giải thích được vì sao P99 quan trọng hơn average, và SLO khác SLA thế nào
- **Apply** — implement được structured logging với correlation ID + PII sanitization cho agent đang deploy
- **Analyze** — đọc được trace waterfall và xác định bottleneck trong agent pipeline nhiều bước
- **Evaluate** — so sánh được Langfuse / LangSmith / Helicone / Phoenix cho use case cụ thể
- **Create** — thiết kế được monitoring blueprint với SLO, alert rules, và dashboard 3 layers

# Deliverable Cuối Ngày

Monitoring blueprint hoàn chỉnh cho deployed agent: structured logging + tracing + dashboard + alert rules dựa trên SLO.

- 1 structured logging pipeline: JSON, correlation ID, PII sanitized, 3 log levels
- 1 Langfuse/LangSmith integration: $\geq$ 10 traces với cost + latency + quality tags
- 1 monitoring dashboard: 4 golden signals + cost + quality (6 panels tối thiểu)
- 1 SLO definition + alert rules: symptom-based, có runbook link
- 1 trang blueprint document: mô tả SLO, architecture diagram, alert playbook

# Foundations of Observability

01

Từ lý thuyết xuống thực hành: observability là gì, và vì sao chỉ monitoring không đủ cho AI agent

# Monitoring vs Observability — Khác Nhau Thế Nào?

VINUNIVERSITY

## Monitoring (biết trước)

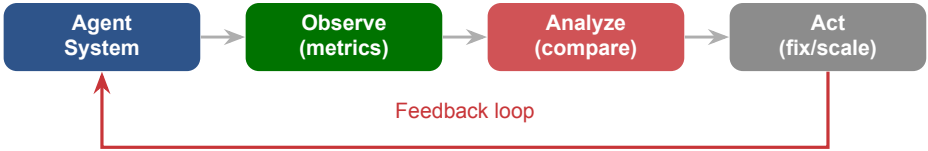
- Đo các metrics **biết trước** sẽ cần
- Trả lời câu hỏi **đã định sẵn**
- Phù hợp với *known-unknowns*
- Ví dụ: CPU $>$ 80%? uptime?

## Observability (khám phá)

- Hệ thống phát ra đủ **data** để hỏi câu hỏi **mới**
- Phù hợp với *unknown-unknowns*
- Ví dụ: vì sao user X với feature Y, model Z, tháng W latency cao?

AI agent có rất nhiều **unknown-unknowns** (prompt mới, model drift, user creativity) $\rightarrow$ observability quan trọng hơn monitoring đơn thuần.

# Control Theory — Vì Sao Quan Trọng




**MTTD** (Mean Time To Detect): từ khi sự cố xảy ra đến khi phát hiện.

**MTTR** (Mean Time To Recover): từ khi phát hiện đến khi fix xong.

> Monitoring tốt = **giảm MTTD xuống phút**, không phải ngày. Không có observability, MTTD = thời gian user phản ánh.

# Từ Artifact Day 12 Sang Production Reality

**Day 12 đã làm được**

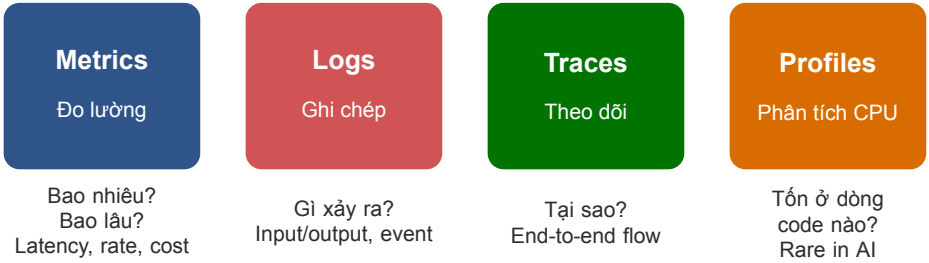
- Agent deployed trên cloud
- Public URL hoạt động
- Health check endpoint
- Basic authentication

**Nhưng chưa trả lời**

- Agent đang chậm hay nhanh? P99?
- Tốn bao nhiêu tiền mỗi user/ngày?
- Bao nhiêu request fail, và loại lỗi gì?
- Câu trả lời có đúng không?
- Khi nào cần scale up?

**Lưu ý:** Không có monitoring, bạn chỉ biết agent hỏng **khi user phàn nàn**. Với AI agent, user thường không phàn nàn — họ **im lặng churn**.

# 3 Pillars of Observability + Profiles




**Metrics** = số liệu aggregate (rẻ, alert). **Logs** = event chi tiết (đắt, debug). **Traces** = hành trình request (vừa phải, root-cause).

Pillar thứ 4 **Profiles** (CPU/memory) ít dùng cho AI vì bottleneck thường ở LLM API, không code local.

# Cardinality — Tại Sao Metrics Đắt

**Cardinality** = số tổ hợp unique của các label.

**Ví dụ:** metric `llm_latency` có labels:

- `user_id`: 100,000 giá trị
- `model`: 4 giá trị
- `endpoint`: 20 giá trị
- `status`: 10 giá trị

Tổng series $= 100{,}000 \times 4 \times 20 \times 10 = \mathbf{8 \times 10^7}$ series!

Với Prometheus, mỗi series $\approx$ 500 bytes/15s $\rightarrow$ $\sim$**40GB/giờ**. Không khả thi.

## Quy tắc cardinality

- **KHÔNG** dùng `user_id`, `request_id`, `email` làm metric label
- **CÓ** dùng: `model`, `env`, `feature`, `tier`
- High-cardinality data $\rightarrow$ vào **logs** hoặc **traces**
- Sampling 1–10% cho AI workloads

**Lưu ý:** Sai cardinality là cách phổ biến nhất làm **sập Prometheus** và nhận bill $40k/tháng từ Datadog không báo trước.

# 3 Pillars Cùng Nhau — Một Ví Dụ Thực Tế

**Kịch bản:** Alert fire lúc 3h sáng: "P99 latency > 8s suốt 10 phút".

1. **Metrics** cho thấy P99 đã tăng từ 2s → 8s trong 10 phút. Error rate vẫn 0%.
   → *Biết có vấn đề, nhưng chưa biết vì sao.*
2. **Traces** cho 10 request chậm nhất: span `vector_search` chiếm 6.5s (bình thường 200ms).
   → *Bottleneck tại retrieval, không phải LLM.*
3. **Logs** tại service retrieval: ``` ``Pinecone request timeout after 6000ms'' ``` bắt đầu từ 02:47.
   → *Root cause: Pinecone có sự cố, cần chuyển sang fallback vector DB.*

> Metrics → phát hiện. Traces → localize. Logs → root cause. Thiếu 1 trong 3, MTTR sẽ kéo dài từ phút lên giờ.

# Cost of Not Monitoring — 4 Rủi Ro

Agent trả lời sai, không ai biết. Đến khi phát hiện thì đã mất user.

Token cost tăng dần không alert. Cuối tháng bill gấp 5 lần dự kiến.

Latency P95 tăng 10ms/tuần. 6 tuần sau: 2x chậm hơn. Không baseline → không ai để ý.

"Agent sai hôm qua." Không log, không trace. Không reproduce → không fix.

**Lưu ý:** **Air Canada 2024:** chatbot tư vấn sai chính sách, tòa buộc trả tiền theo câu chatbot. Không monitoring = không phòng vệ.

# SLI / SLO / SLA — Ngôn Ngữ Của Reliability

| Thuật ngữ | Định nghĩa | Ví dụ cho agent |
|---|---|---|
| **SLI** | Service Level Indicator — 1 metric đo lường được | P95 latency = 2.3s |
| **SLO** | Objective: mục tiêu nội bộ cho SLI | P95 $\leq$ 3s trong 99.5% thời gian (28 ngày) |
| **SLA** | Agreement: cam kết với khách hàng, có phạt | 99% uptime, nếu không sẽ refund 10% |
| **Error Budget** | Dung sai cho phép SLO sai lệch | $100\% - 99.5\% = 0.5\%$ = 3.6 giờ/tháng |

SLO **chặt hơn** SLA (buffer để không bao giờ breach SLA). Khi error budget còn nhiều $\rightarrow$ được phép deploy risky. Hết budget $\rightarrow$ freeze, chỉ fix bug.

# Error Budget Math — Bao Nhiêu Downtime Chấp Nhận?

| SLO | Downtime/tháng | Downtime/tuần | Downtime/ngày |
|---|---|---|---|
| 99% | 7.2 giờ | 1.68 giờ | 14.4 phút |
| 99.5% | 3.6 giờ | 50.4 phút | 7.2 phút |
| **99.9%** (3 nines) | **43.2 phút** | **10.1 phút** | **1.44 phút** |
| 99.99% (4 nines) | 4.32 phút | 1.01 phút | 8.6 giây |
| 99.999% (5 nines) | 25.9 giây | 6.05 giây | 0.86 giây |

**Lưu ý:** Mỗi thêm 1 “nine” tốn gấp 10 lần tiền. AI agent thực tế → SLO **99%–99.5%** là đủ. Đừng hứa 99.99% khi LLM API chỉ 99.9%.

# 4 Golden Signals + 2 Cho AI Agent



**Google SRE — 4 Golden Signals**

1. **Latency** — thời gian phản hồi
2. **Traffic** — request rate (QPS)
3. **Errors** — error rate
4. **Saturation** — tài nguyên còn bao nhiêu?

**AI Agent cần thêm 2**

5. **Cost** — $/request, $/user, token usage
6. **Quality** — hallucination rate, user CSAT, groundedness score

> Agent có thể “up” (traffic OK, latency OK, error OK) nhưng **trả lời sai** và **đốt tiền**. Đây là 2 failure mode riêng của AI mà monitoring truyền thống bỏ qua.

# RED vs USE — Hai Phương Pháp Observability



**RED (request-centric)**

- **R**ate — requests/giây
- **E**rrors — error rate
- **D**uration — latency P50/P95/P99

*Perspective: user — tôi gửi request, được gì?*

**USE (resource-centric)**

- **U**tilization — tài nguyên dùng %
- **S**aturation — có queue/chờ không?
- **E**rrors — lỗi của resource

*Perspective: resource — LLM API, queue đang làm gì?*

Agent chậm (RED: Duration P95 tăng) → debug bằng USE (LLM rate limit utilization 95%) → bị throttle → upgrade tier hoặc fallback.

# AI Metrics Deep Dive

02

AI agent cần metrics riêng: performance, quality, cost, reliability, và drift — những thứ monitoring truyền thống không có

# Performance Metrics — Deep Dive



## Metrics truyền thống

- **Latency** — thời gian end-to-end
- **Throughput** — requests/giây
- **Queue depth** — backlog
- **Uptime** — availability

## AI-specific (mới)

- **TTFT** — Time To First Token
- **TPOT** — Time Per Output Token
- **Tokens/sec** — tốc độ generate
- **Context fill %** — dùng bao nhiêu window

Với streaming UI, user cảm nhận **TTFT**, không phải total latency. TTFT 500ms $\rightarrow$ nhanh kể cả tổng 8s. TTFT 3s $\rightarrow$ lag kể cả tổng 4s. Optimize TTFT trước total.

# Percentile Math — Vì Sao P99 Quan Trọng

**Bài toán:** Agent có P99 $=$ 5s, user chat 10 lượt.
**Xác suất user gặp $\geq$ 1 lần $>$ 5s:**

$$P = 1 - 0.99^{10} \approx \mathbf{9.6}\%$$

- 1/10 user sẽ gặp lag rất tệ
- Với 1,000 user/ngày $\rightarrow$ 96 user bức xúc
- Họ sẽ là người tweet negative, churn, complain

**Amazon Rule**

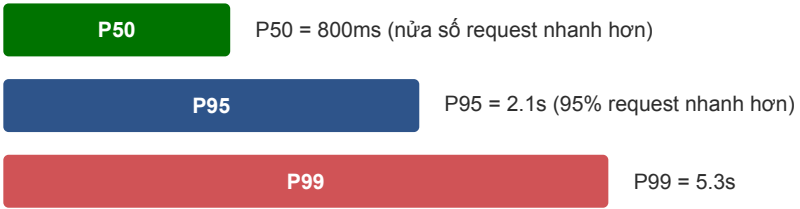
*"Every 100ms of latency cost them 1% of sales."*

$\rightarrow$ Optimize **tail**, không chỉ average.

$\rightarrow$ P99, P99.9 là KPI chính thức tại Amazon, Google, Meta.

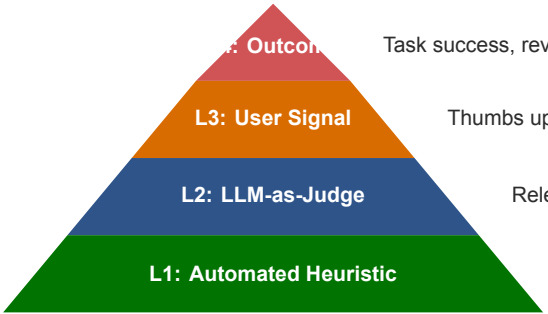
Service có P99 $=$ X ms, user dùng N lượt $\rightarrow$ khả năng gặp tail $= 1 - 0.99^N$. **Tail latency compounds nhanh** trong agentic workflow nhiều bước — 5 bước $\times$ P99 = P95 của toàn bộ!

# Percentile Latency — P50 vs P95 vs P99




**Lưu ý:** Trung bình (average) ẩn giấu long tail. Nếu P50 là 800ms nhưng P99 là 5s, nghĩa là **1 trên 100 user phải chờ rất lâu** — và họ là người dễ churn nhất.

# Quality Metrics — Kim Tự Tháp 4 Tầng




L1 rẻ, realtime nhưng không nói được quality thực. L4 là ground truth nhưng lag hàng tuần. **Production** cần cả 4 — L1/L2 để alert, L3/L4 để confirm trends.

# Hallucination — Phát Hiện Thế Nào?

**Hallucination** = agent trả lời "rất tự tin" nhưng sai sự thật. Không có 1 metric duy nhất → cần combo 4 patterns:

## Pattern 1: Groundedness (RAG)

- Mỗi claim trong output, check có trong retrieved context không
- Tool: RAGAS `faithfulness`, TruLens

## Pattern 2: Self-consistency

- Gọi LLM 3 lần với temp = 0.7
- Nếu 3 câu trả lời mâu thuẫn → nghi ngờ
- Cost $3\times$ → dùng cho sample 1%

## Pattern 3: Entity verification

- Extract entities (tên, số, dates)
- Cross-check với DB/API đáng tin
- Cho finance, medical use cases

## Pattern 4: User feedback loop

- "Was this helpful?" button
- Regenerate click = tín hiệu hallucination
- Signal chậm nhưng rẻ và thật

**Lưu ý:** Air Canada revisited: Chatbot tư vấn chính sách bereavement fare **không tồn tại**. Nếu có groundedness check với actual policy DB → block câu trả lời từ đầu, tránh kiện tụng.

# Cost Engineering — Công Thức Cốt Lõi

**Cost per request** không phải 1 con số đơn giản:

$$\text{Cost}_{req} = \underbrace{\frac{T_{in}}{10^6} \cdot P_{in}}_{\text{input cost}} + \underbrace{\frac{T_{out}}{10^6} \cdot P_{out}}_{\text{output cost}} + \underbrace{\frac{T_{cache}}{10^6} \cdot P_{cache}}_{\text{cache read}}$$

## Ví dụ: Claude Sonnet 4.5 (2026)

- Input: \$3 / M tokens
- Output: \$15 / M tokens
- Cache write: \$3.75 / M
- Cache read: \$0.30 / M (**10x rẻ hơn input**)

## Request điển hình: RAG agent

- System + docs: 8,000 tokens in
- User query: 200 tokens in
- Response: 500 tokens out
- Cost $= 8200 \cdot \frac{3}{10^6} + 500 \cdot \frac{15}{10^6}$
- $=$ \$0.0246 + \$0.0075 $=$ **\$0.032/req**

> Với 100k requests/ngày, cost $=$ \$3,200/ngày $=$ **\$96k/tháng**. Nếu prompt cache hit 80% cho system: giảm còn \$1,100/ngày $=$ tiết kiệm **\$63k/tháng**.

# Cost Attribution — Tiền Đi Đâu?

**Câu hỏi CFO luôn hỏi:** “$50k tháng này, ai tốn?” — bạn phải attribute được.

| Dimension | Tag gắn vào trace | Dùng để... |
|---|---|---|
| Per user | `user_id` | Biết power user, tính pricing |
| Per feature | `feature="summary"` | Prioritize optimization |
| Per model | `model="claude-sonnet-4-5"` | So sánh cost/value các model |
| Per tenant | `tenant_id` | Multi-tenant billing |
| Per env | `env="prod"` | Tách dev/staging noise |
| Per cohort | `plan="enterprise"` | Margin analysis |

Mọi LLM call phải có **3 tags tối thiểu**: `user_id`, `feature`, `model`. Thiếu 1 trong 3 → khi CFO hỏi, bạn không trả lời được, và ngân sách bị cắt.

# 4 Cost Optimization Patterns



### 1. Prompt Caching

Cache system prompt + docs tĩnh. Anthropic: **10x rẻ hơn** cache read. → **giảm 70% cost**

### 2. Model Routing (Cascade)

Dễ → Haiku. Khó → Sonnet. Classifier nhẹ quyết định. → **40–60% giảm cost**

### 3. Semantic Cache

Query tương tự → trả lời cached (embedding similarity $> 0.95$). → **Hit rate 20–40%**

### 4. Batch API

Non-realtime (summary đêm) → batch giá **50%**. → **50% offline work**

Apply cả 4 patterns: \$96k/tháng → \$22k/tháng (77% tiết kiệm). Đây là **FinOps for AI** — kỹ năng hot 2026.

# Reliability Metrics — Error Taxonomy



**"Error rate 5%"** — nhưng loại lỗi gì? Không taxonomy → không fix được.

| Loại lỗi | Nguyên nhân | Cách handle |
|---|---|---|
| LLM API 5xx | Provider down/rate limit | Retry exponential backoff, fallback model |
| LLM timeout | Slow provider, network | Circuit breaker, client timeout $<$ server |
| Tool call failed | External API lỗi | Retry, graceful degradation |
| Tool schema invalid | LLM sinh JSON lỗi | Re-prompt với error feedback |
| Guardrail block | Content policy vi phạm | Log + user-friendly message |
| Empty response | LLM refuse/filter | Alternate prompt, escalate to human |
| Context overflow | Input $>$ limit | Truncate, summarize history |

Track `error_type` field trong mỗi log. Alert fire → biết ngay gọi ai (LLM provider? tool owner? prompt engineer?).

# User-centric Metrics — Ngoài Kỹ Thuật

**Engineering metrics xanh không có nghĩa là user happy.** Cần đo trực tiếp:

## Explicit signals

- **Thumbs up/down** — gắn vào mỗi response
- **CSAT survey** — weekly sample
- **NPS** — quarterly
- **Escalation to human** — rate

→ *Signal rõ, response rate thấp ($< 5\%$)*

## Implicit signals (quan trọng hơn)

- **Regenerate rate** — user bấm "thử lại"
- **Session length** — dùng lâu = hài lòng?
- **Follow-up rate** — câu hỏi tiếp
- **Conversion** — action sau chat
- **Return rate** — quay lại trong 7 ngày

> Họ thấy Regenerate rate là **predictor tốt nhất** cho churn — hơn cả thumbs down. User không bấm thumbs down, họ chỉ... rời đi.

# Drift — Khi Data/Model Thay Đổi Âm Thầm

**3 loại drift cần monitor:**

- **Data drift** — input distribution thay đổi (user hỏi kiểu mới)
- **Concept drift** — mapping input→output thay đổi (luật mới)
- **Model drift** — provider update model, behavior đổi

**Phát hiện bằng:**

- Population Stability Index (PSI)
- KL-divergence giữa distributions
- Embedding drift (cosine similarity)

**PSI formula**

$$\text{PSI} = \sum_i (p_i - q_i) \ln \frac{p_i}{q_i}$$

**Interpret:**

- $< 0.1$ — stable
- 0.1–0.25 — mild drift
- $> 0.25$ — significant, cần retrain

**Lưu ý:** Case 2024: OpenAI silently update GPT-4 → format output đổi → nhiều pipelines breaks in silence. Không drift monitoring = không biết.

# Metric Nào Cho Ai?

| Stakeholder | Quan tâm | Metrics |
|---|---|---|
| Engineering | System health, debug | Latency P95/P99, error rate, tool call failure, saturation |
| Product | User experience | CSAT, task completion, hallucination rate, re-generate rate |
| Finance / Ops | Cost control | Cost/day, tokens/request, cost by model, attribution |
| Leadership | ROI overview | Adoption rate, cost vs value, uptime, user growth |
| Security / Legal | Compliance | PII leak rate, audit log completeness, data retention |

Dashboard cho stakeholders phải nói bằng **ngôn ngữ business**. “P95 = 2.1s” → “95% user nhận trả lời trong 2s”.

# Structured Logging

03

Biến log thành **data có thể query được** — nền tảng cho debug AI agent ở scale

# Unstructured vs Structured Log

```
# --- Unstructured: kho search, kho filter ---
# 2026-03-18 10:23:45 INFO Agent responded to query
# 2026-03-18 10:23:46 ERROR Tool call failed

# --- Structured JSON: de query, de aggregate ---
log_entry = {
    "ts": "2026-03-18T10:23:45Z",
    "level": "INFO",
    "correlation_id": "req-abc123",
    "user_id": "u_7842",
    "event": "agent_response",
    "latency_ms": 1250,
    "tokens_in": 850, "tokens_out": 120,
    "cost_usd": 0.0043,
    "model": "claude-sonnet-4-5",
    "feature": "summary"
}
```

# Log Schema — 3 Tier Design



### Tier 1: Required

Mỗi log entry **phải** có:

- `ts` (ISO 8601)
- `level` (INFO/WARN/ERROR)
- `correlation_id`
- `service` name
- `event` type

### Tier 2: Context

**Nên có** khi liên quan:

- `user_id` (hashed)
- `session_id`
- `feature`
- `model`
- `env`

### Tier 3: Payload

**Event-specific** metrics:

- `latency_ms`
- `tokens_in/out`
- `cost_usd`
- `error_type`
- `tool_name`

Viết thành Pydantic model hoặc JSON Schema, validate trước khi emit. Nếu mỗi service log khác nhau → dashboard không build được.

# Python `structlog` — Setup Chuẩn

```python
import structlog, logging

structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
        structlog.processors.TimeStamper(fmt="iso", utc=True),
        structlog.processors.StackInfoRenderer(),
        structlog.processors.format_exc_info,
        structlog.processors.JSONRenderer(),
    ],
    wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),
    cache_logger_on_first_use=True,
)

log = structlog.get_logger()
log.info("agent_started", service="rag-agent", version="1.2.0")
# --> {"event":"agent_started","level":"info","timestamp":"..."}
```

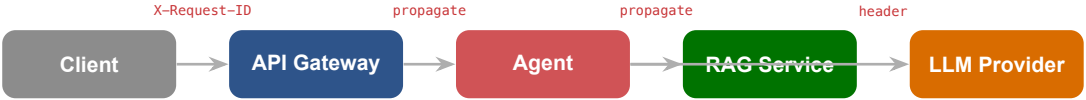
# contextvars — Correlation ID Tự Động

```python
from structlog.contextvars import bind_contextvars, clear_contextvars
import uuid

async def handle_request(request):
    clear_contextvars()
    bind_contextvars(
        correlation_id=str(uuid.uuid4())[:8],
        user_id=request.user_id, feature=request.feature,
    )
    # Tu day tro di, moi log.* tu dong co 3 fields nay
    log.info("request_received", query_len=len(request.query))
    result = await agent.run(request.query)
    log.info("response_sent", latency_ms=result.latency)
    return result
```

`contextvars` an toàn với `asyncio` — mỗi request có context riêng, không bị lẫn giữa

# Correlation ID — Xuyên Suốt Services




```
correlation_id = "req-abc123"
```

Mọi log entry đều có field này → grep 1 ID ra full journey

> HTTP header `traceparent` (W3C Trace Context) = correlation ID chuẩn quốc tế. Mọi framework hiện đại (FastAPI, Express) đều support.

# PII Sanitization — Nên Log Gì Và Không



**Nên log**

- Input **đã sanitize** (độ dài, topic)
- Output summary (không full text)
- Tool calls + results (metadata)
- Latency, tokens, cost
- Errors + stack traces
- Correlation ID, hashed user ID

**KHÔNG log**

- PII (tên, SĐT, CCCD, email)
- Full prompt chứa sensitive data
- API keys, tokens, secrets
- Credit card, bank account
- Full raw user input
- DEBUG verbose ở production

**Lưu ý:** Log PII = vi phạm GDPR, PDPA, CCPA. Phạt có thể **4% revenue toàn cầu** (GDPR). Sanitize **trước** khi log, không phải sau khi bị audit.

# PII Scrubbing — Regex + Presidio

```python
# --- Cach 1: regex nhanh cho common patterns ---
import re
PII_PATTERNS = {
    "email": r"[\w\.-]+@[\w\.-]+\.\w+",
    "phone_vn": r"(\+84|0)\d{9,10}",
    "cccd": r"\b\d{12}\b",
    "credit_card": r"\b\d{4}[- ]?\d{4}[- ]?\d{4}[- ]?\d{4}\b",
}
def scrub(text):
    for n, p in PII_PATTERNS.items():
        text = re.sub(p, f"[REDACTED_{n.upper()}]", text)
    return text

# --- Cach 2: Microsoft Presidio (NER-based, robust) ---
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine
analyzer, anonymizer = AnalyzerEngine(), AnonymizerEngine()
results = analyzer.analyze(text=user_input, language="en")
safe = anonymizer.anonymize(text=user_input, analyzer_results=results).text
```

# Log Levels — Khi Nào Dùng

| Level | Khi nào dùng | Ví dụ | Prod? |
|---|---|---|---|
| `DEBUG` | Dev only, rất chi tiết | Full prompt, intermediate state | Không (sample 1%) |
| `INFO` | Normal flow, milestone events | Request received, response sent | Có |
| `WARN` | Degraded nhưng vẫn hoạt động | Retry succeeded, fallback used | Có |
| `ERROR` | Failed, cần attention | Tool call timeout, LLM error | Có, alert |
| `CRITICAL` | System-level failure | DB unreachable, all models down | Có, page |

Production chạy **INFO**. Debug cụ thể: tạm bật DEBUG cho request ID đó, xong tắt. Không để DEBUG luôn-on → log volume 100x, bill lên trời.

# Log Sampling — Khi Volume Quá Cao

**Vấn đề:** 100k requests/ngày $\times$ 10 log entries/request $=$ 1M entries/ngày. Elastic/Datadog tính \$0.10/1k entries $\rightarrow$ \$100/ngày chỉ riêng log. Không scalable.

**Strategies**

- **Head sampling** — quyết định sample ngay khi trace bắt đầu (rẻ, có thể miss errors)
- **Tail sampling** — quyết định sau khi trace xong (đắt, giữ được 100% errors)
- **Reservoir sampling** — giữ N mẫu uniform

**Strategy điển hình cho AI agent**

- 100% ERROR và WARN
- 10% INFO (cho normal requests)
- 1% DEBUG (cho deep debug)
- 100% requests $>$ 10s (tail-sampling on latency)
- 100% cost $>$ \$1/request (outliers)

Sampling giảm cost 10–100x, nhưng mất visibility vào normal pattern. Giữ 100% errors là non-negotiable — đó là data debug quan trọng nhất.

# Log Aggregation Stacks

| Stack | Components | Khi nào dùng | Cost/tier |
|---|---|---|---|
| **ELK** | Elasticsearch, Logstash, Kibana | Full-text search mạnh, complex queries | Tự host, OSS |
| **Loki** | Loki, Promtail, Grafana | Label-based (giống Prometheus), rẻ | Tự host, OSS |
| **Datadog Logs** | SaaS | Setup nhanh, alert tốt, đắt ở scale | $0.10/GB |
| **CloudWatch** | AWS native | Đã ở AWS, tích hợp IAM | $0.50/GB ingest |
| **BigQuery** | GCP DW | Analytics SQL, long retention | $0.02/GB scan |

**Langfuse** tự là log store cho LLM calls + SaaS free tier. Dev local → stdout JSON + `jq` là đủ. Scale up mới cần ELK/Loki.

# Audit Log — Tách Biệt Với App Log

**Audit log** = record **who did what when** cho compliance, legal, security.

## App log

- Mục đích: debug, performance
- Retention: 30–90 ngày
- Có thể sample
- Có thể sửa/xóa
- Truy cập: dev team

## Audit log

- Mục đích: compliance, forensics
- Retention: **2–7 năm** (tùy ngành)
- **Không sample** — 100%
- **Append-only** — không sửa được
- Truy cập: restricted (compliance officer)

**Lưu ý:** Sai lầm phổ biến: trộn audit log vào app log → khi cần investigate bị thiếu data. Tách riêng từ ngày đầu: S3 bucket với Object Lock, hoặc dedicated audit service.

# Distributed Tracing

04

Trace cho biết toàn bộ hành trình của 1 request qua nhiều bước — công cụ số một để debug AI agent pipeline

# Trace, Span, Context — Terminology



## Khái niệm cốt lõi

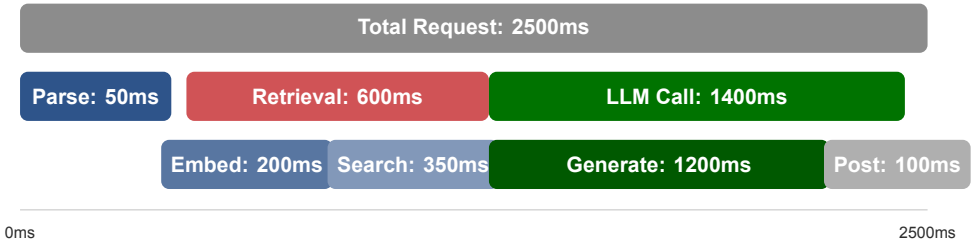
- **Trace** — toàn bộ request end-to-end, có `trace_id` duy nhất
- **Span** — 1 đơn vị công việc trong trace, có `span_id`, `parent_span_id`
- **Context propagation** — cơ chế truyền `trace_id` qua boundaries (HTTP headers, queue messages)

## Span có gì?

- `name` — tên operation (e.g. `llm.generate`)
- `start_time`, `duration`
- `attributes` — key-value (model, tokens)
- `status` — OK / ERROR
- `events` — logs gắn vào span
- `links` — related spans (e.g. retry)

> Trace = cây. Root span là entry point (HTTP request). Child spans là các bước con (RAG retrieve, LLM call, tool call). Nhìn cây biết bottleneck.

# Trace Là Gì — Waterfall View




Mỗi hàng ngang là 1 **span**. Tất cả spans của 1 request tạo thành 1 **trace**. Nhìn trace biết ngay **bottleneck ở đâu**.

# OpenTelemetry — Tiêu Chuẩn Mở

**OpenTelemetry (OTel)** = CNCF graduated project (2024), vendor-neutral tiêu chuẩn cho observability.

### OTel gồm 3 phần

- **API** — interface để instrument code (ngôn ngữ-agnostic)
- **SDK** — implementation cho ngôn ngữ cụ thể (Python, Go, Java...)
- **Collector** — receive, process, export data đến backend (Jaeger, Tempo, Datadog...)

### Vì sao quan trọng

- Instrument **1 lần** — switch backend tự do (khỏi vendor lock-in)
- Auto-instrumentation cho FastAPI, requests, SQLAlchemy...
- 2024: OTel phát hành **GenAI Semantic Conventions** cho LLM tracing

> Học OTel một lần → dùng được Langfuse, Datadog, Jaeger, New Relic đều được. Tất cả đều accept OTLP protocol.

# OpenTelemetry Python — Manual Instrumentation

```python
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter

# Setup 1 lan o startup
provider = TracerProvider()
provider.add_span_processor(BatchSpanProcessor(OTLPSpanExporter()))
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("agent-service")

# Instrument function
def rag_retrieve(query):
    with tracer.start_as_current_span("rag.retrieve") as span:
        span.set_attribute("rag.query_len", len(query))
        results = vector_db.search(query)
        span.set_attribute("rag.result_count", len(results))
        return results
```

# OTel GenAI Semantic Conventions

**Tiêu chuẩn từ 2024:** mọi LLM trace **nên** dùng các attribute names này → tool vendor-agnostic hiểu được.

| Attribute | Ý nghĩa |
|---|---|
| `gen_ai.system` | Provider: “anthropic”, “openai”, “google” |
| `gen_ai.request.model` | Model name: “claude-sonnet-4-5” |
| `gen_ai.request.temperature` | Sampling temp |
| `gen_ai.request.max_tokens` | Max tokens cap |
| `gen_ai.usage.input_tokens` | Input tokens |
| `gen_ai.usage.output_tokens` | Output tokens |
| `gen_ai.response.finish_reasons` | “stop”, “length”, “tool_use” |
| `gen_ai.operation.name` | “chat”, “completion”, “embeddings” |

Dùng namespace `gen_ai.*` → dashboard queries reusable. Tránh custom names như `my_model` — không interoperable.

# Auto-instrumentation — Ít Code Hơn

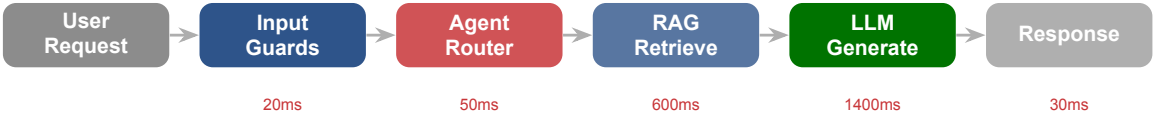
```
# --- Install ---
# pip install opentelemetry-distro opentelemetry-exporter-otlp \
#   opentelemetry-instrumentation-fastapi opentelemetry-instrumentation-anthropic

# --- Run (khong can sua code) ---
# opentelemetry-instrument --traces_exporter otlp uvicorn app:main

# --- app.py: code thong thuong, khong co OTel boilerplate ---
from fastapi import FastAPI
from anthropic import Anthropic
app = FastAPI(); client = Anthropic()

@app.post("/chat")
async def chat(msg: str):
    r = client.messages.create(model="claude-sonnet-4-5",
        max_tokens=1024, messages=[{"role":"user","content":msg}])
    return {"reply": r.content[0].text}
# Moi HTTP request + LLM call tu dong tao span
```

# Trace Cho Agent Pipeline




LLM Generate chiếm 67% latency. Muốn tối ưu → bắt đầu **từ LLM call**: model nhỏ hơn, prompt ngắn hơn, streaming hoặc parallelize RAG với LLM warmup.

# 4 Bottleneck Patterns Trong Trace



### Pattern 1: Sequential dependency

- A $\rightarrow$ B $\rightarrow$ C, tổng $=$ sum
- Fix: parallelize A, B nếu không phụ thuộc

### Pattern 2: N+1 queries

- Loop gọi API/DB nhiều lần
- Trace: nhiều span ngắn cùng tên
- Fix: batch API, hoặc pre-fetch

### Pattern 3: Waiting (không phải CPU)

- Span dài nhưng CPU idle
- LLM API, DB, network
- Fix: parallelize, cache, timeout

### Pattern 4: Retry storm

- Nhiều span retry trong 1 trace
- Backoff quá ngắn, không jitter
- Fix: exponential backoff, circuit breaker

> Nhìn span dài nhất $\rightarrow$ hỏi “có parallelize được không?”. Nhìn nhiều span ngắn lặp $\rightarrow$ hỏi “có batch được không?”. Đó là 80% tối ưu hóa.

# Tool Comparison — 4 Lựa Chọn Hàng Đầu



| Tool | Strength | Best for | Pricing | OSS? |
|---|---|---|---|---|
| **Langfuse** | Full stack: tracing, eval, prompts, cost | Self-host, framework-agnostic | Free tier, cloud paid | Yes (MIT) |
| **LangSmith** | Tight LangChain, playground, datasets | LangChain users | Free tier, seat-based | No (SaaS) |
| **Helicone** | Proxy approach, 0 code change | Rapid integration | Free tier, usage-based | Yes (Apache) |
| **Arize Phoenix** | OpenTelemetry-native, OSS | Research, eval workflows | Free OSS | Yes (Apache) |

Chọn **Langfuse** — open source, free cloud tier, framework-agnostic, đã support OTel GenAI conventions. UI tốt nhất cho LLM-specific features.

# Langfuse Integration — 5 Dòng Code

```python
from langfuse.decorators import observe, langfuse_context
from anthropic import Anthropic
client = Anthropic()

@observe()
def rag_agent(query: str, user_id: str):
    langfuse_context.update_current_trace(user_id=user_id, tags=["prod","rag"])
    docs = retrieve(query)
    response = client.messages.create(
        model="claude-sonnet-4-5", max_tokens=1024,
        messages=[{"role":"user","content":f"Docs: {docs}\n\nQ: {query}"}])
    langfuse_context.update_current_observation(
        metadata={"doc_count": len(docs)},
        usage_details={"input": response.usage.input_tokens,
                       "output": response.usage.output_tokens})
    return response.content[0].text
```

Latency, input/output, cost (model pricing built-in), trace waterfall. Xem ngay trên

# Case Debug Thực Tế — Dùng Trace Để Tìm Bottleneck

**Vấn đề:** User phàn nàn agent chậm "đôi khi". Metrics nói P95 = 2.3s, OK. P99 = 12s — không OK.

1. **Filter trace** theo `duration > 10s` trong 24h qua → được 47 traces
2. **Nhóm theo feature tag** → 42/47 thuộc feature `"legal_qa"`
3. **Mở waterfall** 1 trace: LLM span = 800ms (OK), RAG span = 10.2s (!)
4. **Drill vào RAG span:** sub-span `vector_search` = 9.8s với query length 4,200 tokens
5. **Root cause:** user dán full document vào query → embedding gọi với input lớn → chậm
6. **Fix:** truncate query to 500 tokens before embedding, thêm warning cho user

> Không có trace → chỉ biết "đôi khi chậm", không biết bước nào, không biết pattern nào. Với trace: **10 phút** từ bug report → root cause.

# SLO-based Alerting & Dash-board

05

Metrics và traces chỉ có giá trị nếu có người nhìn — học design alert chất lượng dựa trên SLO

# Symptom vs Cause — Alert Về Cái Gì?

**Cause-based (sai)**

Alert về nguyên nhân kỹ thuật:

- CPU $>$ 80%
- RAM $>$ 90%
- Disk $>$ 85%
- LLM API retry count $>$ 5

*Cause cao nhưng user không bị ảnh hưởng → false positive. Ignore dần.*

**Symptom-based (đúng)**

Alert về impact tới user:

- P95 latency $>$ SLO
- Error rate $>$ 1%
- Cost/hour $>$ budget
- Hallucination rate spike

*Chỉ alert khi user thực sự bị ảnh hưởng. Ít false positive.*

> *"Page on symptoms, not causes."* Cause alerts chỉ là supplementary info để debug sau khi symptom alert fire.

**Vấn đề:** Alert đơn giản "error rate $>$ 1% trong 5 phút" $\rightarrow$ fire quá nhanh (noise) hoặc quá chậm (miss incident). **Giải pháp của Google:** kết hợp 2 windows với 2 burn rates:

| Severity | Short window | Long window | Burn rate (vs SLO) | Ý nghĩa |
|---|---|---|---|---|
| Page (critical) | 5 phút | 1 giờ | 14.4x | |
| Ticket (warn) | 30 phút | 6 giờ | 6x | |

**"14.4x burn rate":** nếu error giữ mức này, tháng sẽ burn hết error budget trong 2 ngày thay vì 30.

Alert fire khi **cả 2 window** cùng vượt threshold. Short window $\rightarrow$ react nhanh với spike thực. Long window $\rightarrow$ filter noise ngắn. Google SRE Workbook Chapter 5.

# Alert Anatomy — Alert Tốt Có Gì?

**Template cho mỗi alert:**

- **Title rõ ràng:** "`[P1] Agent P95 latency > 5s cho feature=summary`"
- **Severity:** P1 (page ngay) / P2 (trong giờ hành chính) / P3 (ticket)
- **Impact statement:** "5% user đang bị chậm > 5s"
- **Current value:** "P95 = 6.3s, bình thường 1.8s, threshold 5s"
- **Dashboard link:** pre-filtered với feature tag, last 1h
- **Trace link:** top 10 traces chậm nhất trong window
- **Runbook link:** playbook fix step-by-step
- **On-call owner:** team/người chịu trách nhiệm

**Lưu ý:** Alert không có runbook = alert không thể xử lý lúc 3h sáng. Viết runbook là **một phần** của work "tạo alert", không phải nice-to-have.

# Alert Rules Cho AI Agent — Bộ Tối Thiểu

| Metric | Threshold | Severity | Channel + Action |
|---|---|---|---|
| Latency P95 | $>$ 5s, 30 phút | P2 Warning | Slack, investigate trace |
| Error rate | $>$ 5%, 5 phút | P1 Critical | Slack $+$ PagerDuty, rollback |
| Hourly cost | $>$ \$50 (budget $\times$ 2) | P2 Warning | Email, check users/features |
| Daily cost | $>$ daily budget | P1 Critical | Email $+$ SMS, kill switch |
| Tool call failure | $>$ 10%, 15 phút | P2 Warning | Slack, check tool provider |
| Hallucination rate | $>$ baseline $+2\sigma$ | P2 Warning | Slack, review outputs |
| Uptime | $<$ 99%, 1 giờ | P1 Critical | PagerDuty, incident response |

Alert phải **actionable**. Không biết phải làm gì $\rightarrow$ redesign hoặc bỏ. Alert nào không fire 90 ngày $\rightarrow$ bỏ.

# Alert Fatigue — Kẻ Giết Hệ Thống Alert



**Alert fatigue xảy ra khi**

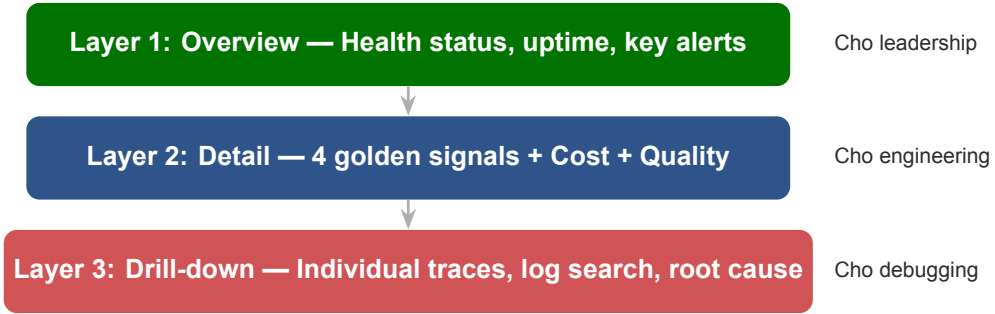
- quá nhiều alerts không quan trọng
- nhận alert 24/7, ngay cả đêm
- mọi người bắt đầu ignore
- alert thật bị lẫn trong noise
- team mất tin tưởng vào hệ thống

**Cách tránh**

- chỉ alert khi cần **hành động ngay**
- phân severity rõ ràng (P1/P2/P3)
- route đúng người, đúng channel
- review và tuning alerts hàng tuần
- auto-resolve khi metric recovery
- dedup trong window

**Lưu ý:** Nếu team ignore alerts thường xuyên, hệ thống alerting đang **tệ hơn không có alert** — alert thật sẽ bị miss. Ít alert chất lượng tốt hơn nhiều alert vô nghĩa.

# Dashboard Design — 3 Layers




Mỗi stakeholder chỉ cần nhìn 1 layer. Leadership cần overview, không cần trace. Engineer cần drill-down, không cần revenue chart. Cùng 1 dashboard cho tất cả → không ai nhìn.

# Dashboard Layer 2 — Bố Cục Chuẩn




Nhiều panel hơn → không ai đọc. Giới hạn 6 panels ở layer 2 → nhìn 5 giây biết health status. Muốn chi tiết hơn → click vào layer 3.

# Dashboard Tools Landscape 2026

Open source, powerful.

Kết nối mọi data source (Prom, Loki, CloudWatch).

**Khi nào:** team có infra ops, muốn kiểm soát

All-in-one SaaS.

Setup nhanh, alerting tốt, đắt ở scale.

**Khi nào:** cần nhanh, có budget, muốn 1 tool

LLM-native hoặc Python custom.

Full control, dễ customize.

**Khi nào:** AI agent MVP, demo, PoC

**Lưu ý:** Cho lab: **Langfuse dashboard** đã đủ cho MVP. Đừng dành thời gian build Grafana custom trước khi có đủ data — YAGNI principle.

# Dashboard Anti-patterns — 5 Điều Nên Tránh



1. **“Wall of metrics”** — 30 panels, không ai nhìn hết. Giới hạn 6–8 panels/layer.
2. **Time range mặc định quá dài** — default 1 giờ cho ops dashboard, không phải 1 tháng (che mất spike)
3. **Không có baseline/threshold line** — nhìn số P95 = 2.1s không biết tốt hay xấu. Luôn vẽ đường SLO lên chart.
4. **Metric không có đơn vị/context** — “Cost: 1250” là gì? USD? ngày? Luôn label đầy đủ.
5. **Không auto-refresh** — dashboard cần realtime (15–30s refresh) cho ops. Cho monthly report thì khác.

> Đưa dashboard cho người không trong team xem 30 giây → họ có nói được “hệ thống đang OK” hay “đang có vấn đề ở X” không? Nếu không, redesign.

# Tools Landscape & Case Studies

06

Chọn tool đúng tiết kiệm tháng setup. Học từ case thực: Notion AI, Replit, và 7 anti-patterns từ industry

# LLM Observability Landscape 2026

| Category | Leader 2026 | Strength | Watch |
|---|---|---|---|
| LLM-native | Langfuse, LangSmith | Prompt, eval, cost | Helicone, Phoenix |
| APM truyền thống | Datadog, New Relic | Infra + app + AI | Honeycomb, Lightstep |
| OSS self-host | Jaeger, Tempo, Loki | Zero vendor lock | SigNoz |
| Cost-focused | Helicone, OpenMeter | Token accounting | Vellum |
| Eval-focused | Arize Phoenix, Patronus | Quality & drift | Galileo |

Converge: APM truyền thống (Datadog) add LLM features. LLM-native (Langfuse) add infra tracing. Trong 12 tháng tới, **OpenTelemetry GenAI semconv** thành chuẩn chung.

# Decision Framework — Chọn Tool Thế Nào?



### Q1: Team size?

- 1–5: SaaS, free tier
- 5–50: SaaS paid
- 50+: Hybrid / self-host

### Q2: Compliance?

- HIPAA/PCI: self-host
- GDPR: EU region SaaS
- None: bất kỳ

### Q3: Existing stack?

- Datadog: stay, add AI addon
- LangChain: LangSmith
- Agnostic: Langfuse

### Q4: Budget/month?

- $0: Langfuse cloud free
- $100–500: LangSmith/Helicone
- $500+: Datadog full stack

### Q5: Skill set?

- Python-heavy: Langfuse
- Infra ops: Grafana
- Non-dev: Datadog

### Q6: Evaluation?

- Quality cần: Phoenix, LangSmith
- Cost only: Helicone
- Full stack: Langfuse

> Không có “best tool”. Có best tool **cho team + use case + budget của bạn**. Đừng copy stack của FAANG — họ có infra team 50 người.

# Case Study 1 — Notion AI Cost Optimization



**Bối cảnh:** Notion AI phục vụ hàng triệu user với nhiều features (summary, Q&A, writing assist). Cost OpenAI ban đầu ∼30% revenue. **Monitoring insight:**

- 70% queries là “summarize” với prompt giống nhau
- 15% user chiếm 60% cost (power users với doc dài)
- Regenerate rate cao ở feature “writing assist”

## Actions taken (theo thứ tự ROI):

1. Prompt cache cho system prompt (tất cả features) → giảm 40% input cost
2. Route “summary” qua model nhỏ hơn (Haiku tier) → giảm 60% cost cho feature này
3. Per-user rate limit cho free tier → limit abuse power users
4. Improve prompt cho “writing assist” → giảm regenerate 35%

Total cost / MAU giảm **58%** trong 3 tháng, không giảm quality metrics. Điều làm được vì có monitoring chi tiết theo feature + user + model.

# Case Study 2 — Replit Quality Debugging



**Bối cảnh:** Replit AI code assistant, user báo “suggestions bị tệ hơn tuần trước”. Engineering metrics đều xanh. **Monitoring insight:**

- Regenerate rate tăng từ 12% $\rightarrow$ 23% trong 2 tuần (implicit signal!)
- Thumbs-down rate tăng chậm hơn (user ít bấm)
- Đặc biệt cho ngôn ngữ `python` và `javascript`

**Investigation (bằng trace):**

- Filter trace cho regenerate cases $\rightarrow$ context truncation happening
- Thay đổi tokenizer version tuần trước $\rightarrow$ đếm tokens khác 10%
- Truncation threshold không được update $\rightarrow$ cut off giữa câu

> Revert tokenizer, thêm integration test cho token counting. Lesson: **implicit signals** (regenerate) là early warning, explicit signals (thumbs down) quá chậm. Cả 2 đều cần.

# 7 Anti-patterns Từ Industry

1. **"We'll add monitoring later"** — later = never. Add ngay từ MVP.
2. **Log full prompts và responses** — vi phạm GDPR, storage bill lên trời. Sanitize + sample.
3. **Alert trên mọi metric "quan trọng"** — 50 alerts → alert fatigue → ignore.
4. **Không có runbook** — alert fire lúc 3h sáng, engineer trẻ lost, escalate lên senior.
5. **Monitoring dev ≠ prod config** — prod có issue không reproduce được vì dev khác setup.
6. **Chỉ đo performance, quên cost** — đến cuối tháng mới biết đốt tiền.
7. **Trust vendor mặc định** — LangChain default telemetry có thể log sensitive data. Đọc docs trước khi deploy.

**Lưu ý:** Anti-pattern #1 là phổ biến nhất và tai hại nhất. Monitoring không phải là feature phụ → là phần **core** của production system, ngang với authentication.

# Lab 13 & Closing

07

Monitoring đầy đủ cho agent: biết nó chạy thế nào mà không cần hỏi user, và có blueprint bàn giao được

> Gắn monitoring stack đầy đủ vào deployed agent từ Day 12: structured logging + tracing + dashboard + SLO-based alerts, với blueprint document bàn giao được.

## Phase A: Instrumentation (30 phút)

1. Setup Langfuse project (free cloud tier), lấy API keys
2. Add structlog cho structured JSON logging + correlation ID với contextvars
3. Integrate Langfuse `@observe()` decorator cho 3 functions chính
4. Add tags: `user_id`, `feature`, `model`, `env`

## Phase B: Dashboard + Alerts (30 phút)

5. Define SLO: P95 latency, error rate, daily cost budget
6. Build dashboard Layer 2 (6 panels tối thiểu)
7. Configure 3 alert rules với runbook links

## Phase C: Validation (30 phút)

8. Gửi 10–20 requests variety → check traces xuất hiện
9. Inject 1 failure (bad prompt) → verify alert fire
10. Viết blueprint document (1 trang: SLO, architecture, playbook)

# Lab 13 Troubleshooting — Lỗi Thường Gặp

| Triệu chứng | Cách xử lý |
|---|---|
| Trace không xuất hiện ở Langfuse | Check API key env var, network, bật `LANGFUSE_DEBUG=true` |
| Cost không được track | Model name sai (phải khớp Langfuse pricing DB) hoặc thiếu `usage_details` |
| Correlation ID không propagate qua async | Dùng `contextvars`; kiểm tra `clear_contextvars` ở đầu mỗi request |
| PII leak vào log dù đã scrub | Check order processors: scrubber phải chạy **trước** JSONRenderer |
| Alert fire nhưng không Slack | Check webhook URL, quota rate limit, severity routing rules |
| Dashboard trống dù có traces | Time range filter, tag filter, hoặc data source chưa kết nối |

Bật `DEBUG` cho `langfuse` và `opentelemetry` logger → thấy trace export request/response, tìm ra vấn đề trong 5 phút.

# Blueprint Rubric — Chấm Điểm Deliverable



### Logging & Tracing (40%)

- Structured JSON logs (10)
- Correlation ID propagate (10)
- PII sanitized (10)
- 10+ Langfuse traces (10)

### Dashboard & Alerts (40%)

- 6+ panels Layer 2 (15)
- SLO defined (5)
- 3 alert rules với runbook (15)
- Screenshot có data (5)

### Blueprint document (20%)

- SLO table (5)
- Architecture diagram (5)
- Alert playbook (5)
- Cost & scaling plan (5)

**Điểm bonus (+10)**

- Cost optimization (prompt cache)
- Quality metric (heuristic-based)
- OTel auto-instrument HTTP

# Tổng kết — Key Takeaways

**Những ý chính cần nhớ** trước khi sang bài tiếp theo

1. **3 pillars + 2 extras.** Metrics, Logs, Traces + **Cost** và **Quality** riêng cho AI.
2. **SLO-driven, symptom-based alerting.** Alert khi user bị ảnh hưởng, luôn có runbook.
3. **OpenTelemetry + FinOps.** Instrument 1 lần; 4 cost patterns (cache/routing/semantic/batch) giảm 70%.

# Tiếp theo & Bài tập

**AI Evaluation & Benchmarking**

*"Sếp hỏi: AI agent tốt hơn ChatGPT bao nhiêu? Bạn nói sao nếu không có benchmark? Monitoring đo **đang hoạt động thế nào**, evaluation đo **có tốt không**."*

- Chuẩn bị: 10 câu hỏi mẫu với expected answers cho agent, cover các edge cases
- Đọc trước: RAGAS documentation — `faithfulness`, `answer_relevancy`, `context_precision` (20 phút)
- Suy nghĩ: quality metric nào quan trọng nhất cho use case của bạn? Vì sao?

# Tài Liệu Tham Khảo


1. Google SRE Workbook, *Alerting on SLOs* — sre.google/workbook. Multi-window multi-burn-rate alerting, error budget.
2. Langfuse, *Open Source LLM Observability* — langfuse.com. Tracing, cost tracking, prompt management, eval.
3. LangSmith Documentation — docs.smith.langchain.com. Tracing, evaluation, datasets, monitoring.
4. OpenTelemetry, *GenAI Semantic Conventions* — opentelemetry.io/docs/specs/semconv/gen-ai. Vendor-neutral standard cho LLM tracing.
5. Microsoft Presidio — microsoft.github.io/presidio. PII detection & anonymization SDK.
6. Charity Majors, *Observability Engineering* (O'Reilly 2022) — quyển sách nền tảng về observability hiện đại.
7. Anthropic, *Prompt Caching Best Practices* — docs.anthropic.com/prompt-caching. Giảm 70% cost.

# Hỏi & Đáp

*Monitoring tốt nghĩa là bạn biết agent có vấn đề **trước khi** user phàn nàn — và có đủ data để fix **trong phút**, không phải giờ.*